AI Coding Workshop — Matt Pocock

Conference · 2025 · 2 hours

> *Software engineering fundamentals that work with humans also work exceptionally well with AI, and understanding LLM constraints allows developers to structure work for optimal results.*

LLM Constraints: Smart Zone vs Dumb Zone

- LLMs have a 'smart zone' (early context, 0-100K tokens) where they perform best, and a 'dumb zone' (beyond 100K) where performance degrades as attention relationships strain quadratically
- Size tasks to stay within the smart zone rather than letting context bloat into the dumb zone through continuous additions
- Break large tasks into small phases that each fit in the smart zone, then loop over them (phase N) rather than using multi-phase plans
- Clearing context resets to the system prompt (like Memento) — prefer this over compacting, which creates sediment and maintains degraded state
- Keep system prompts tiny (not 250K tokens) to avoid starting in the dumb zone before doing any work

Every time you add a token to an LLM, it's like adding a team to a football league — the number of matches scales quadratically.

Alignment Through Grilling

- Use a 'grill me' skill to reach shared understanding before implementation — the AI interviews you relentlessly about every aspect of the plan
- This creates a 'design concept' (Frederick P. Brooks) shared between human and AI, preventing misalignment
- Can last 40-100 questions, taking up to an hour, but produces a conversation history documenting the design concept
- Works for pair/mob programming with multiple humans and AI, or for validating meeting transcripts with domain experts
- Prevents the 'specs to code' trap where you ignore code quality and just keep editing specs — code is your battleground

I needed to reach a shared understanding, not just get a plan. I needed to be on the same wavelength as my agent.

PRD and Task Breakdown

- Convert grilling session into a Product Requirements Document (PRD) that defines the destination — what success looks like
- Create a Kanban board of parallelizable issues with blocking relationships, defining the journey
- Use 'trace bullets' — vertical slices through the entire stack that prove the concept end-to-end before building everything
- Each issue should be independently implementable in the smart zone, with clear acceptance criteria
- Don't over-optimize the PRD — alignment happens in grilling, and refinement happens in QA, not in endless PRD iteration

Implementation: Ralph Loop and TDD

- Use a 'Ralph loop' (inspired by Ralph Wiggum) — make small changes that incrementally move toward the PRD destination
- Implement test-driven development (TDD) with red-green-refactor to prevent AI from cheating on tests
- TDD forces AI to instrument code before implementing, making it harder to write tests that just validate bad code
- Run feedback loops (tests, type checks, linters) after every change — quality of feedback loops determines AI output quality
- Clear context between review and implementation so review happens in the smart zone, not after implementation exhausted it

If your code base doesn't have feedback loops, you're never going to get decent output from AI. That is the ceiling.

Deep Modules vs Shallow Modules

- Shallow modules (many small files with small interfaces) are hard for AI to navigate and test — unclear where to draw test boundaries
- Deep modules (small interface, lots of functionality inside) are easy to test and give AI clear boundaries to work within
- Design module interfaces yourself, then delegate implementation to AI — keeps code base comprehensible while moving fast
- Use 'improve code base architecture' skill to scan for opportunities to deepen modules and improve testability
- Prevents losing sense of your code base while moving fast — you understand shapes and interfaces, AI handles internals

Design the interface for these modules, but delegate the implementation. These modules become gray boxes — you know the shape, but can delegate what's inside.

Documentation and Code Review

- Avoid doc rot — don't keep PRDs in repo after implementation, as they become outdated and mislead future AI sessions
- Use GitHub issues marked as closed for historical reference without polluting active context
- QA manually to impose taste and catch issues automated testing misses — prevents 'slop' from fully automated workflows
- Push coding standards to reviewers (always include them), but let implementers pull them (available when needed)
- Use stronger models (Opus) for review, lighter models (Sonnet) for implementation to optimize cost and quality

Parallelization and Workflow

- Human-in-the-loop tasks (planning, alignment, QA) require human presence and cannot be automated away
- AFK tasks (implementation, testing, merging) can run unattended in parallel across multiple agents
- Use tools like Sandcastle to run multiple agents in isolated Git branches, then merge results
- Planner chooses parallelizable issues from backlog, implementers work in sandboxes, reviewer checks commits, merger integrates
- QA phase creates more issues for the Kanban board while implementation continues, enabling continuous refinement

There are two types of tasks in the AI age: human-in-the-loop tasks where a human needs to sit there, and AFK tasks where the human can be away from the keyboard.

Safety and Best Practices

- Monitor token usage constantly with a status line showing exact count — essential for knowing proximity to dumb zone
- Treat AI sessions as having phases: system prompt → exploration → implementation → testing, resetting to prompt on clear
- Default to action for small changes, but read code and plan for multi-file or unfamiliar changes
- Use sub-agents for deep research to preserve main context for implementation — they report summaries back up
- After context compaction, re-confirm position by checking file states rather than relying on memory

You need to know exactly how many tokens you're using so you know how close you are to the dumb zone. Absolutely essential.